

■ PYTHON FROZENSET METHODS — COLOURFUL HANDWRITTEN NOTES

Classroom + Real-Time IT Examples + Practice Labs

1. What is a Frozenset?

Definition

A **frozenset** is an immutable version of a Python set. It contains **unique** values, but after creation you cannot add, remove or change elements.

```
servers = frozenset({"web01", "web02", "db01"})
print(servers)
print(type(servers))
```

Simple idea

set = unique + changeable. **frozenset** = unique + NOT changeable.

2. Why do we use Frozenset?

Real-time use

Use frozenset when a group of values must remain fixed. Examples: approved security groups, fixed permissions, allowed ports, immutable configuration values, or a key used inside a dictionary.

```
allowed_ports = frozenset({80, 443, 22})

# allowed_ports.add(8080)
# ■ AttributeError: frozenset has no add()
```

3. Important Frozenset Properties

Property	Meaning	Example
Unique	Duplicates are removed	<code>frozenset([1,2,2,3])</code>
Immutable	Cannot add/remove/change	<code>fs.add(10)</code> ■
Unordered	No normal indexing	<code>fs[0]</code> ■
Hashable	Can be used as a dictionary key	<code>config = {fs: 'approved'}</code>
Set operations	Can compare/combine sets	<code>fs1 & fs2</code>

4. Frozenset Methods ■

Frozenset does **not** have `add()`, `update()`, `remove()`, `discard()`, `pop()`, `clear()` or similar mutating methods. It supports the non-mutating set comparison methods.

union()

Combines unique values from two or more sets.

```
prod = frozenset({"web01", "web02"})
dr = frozenset({"web02", "web03"})
all_servers = prod.union(dr)
print(all_servers)
```

Real-time use

Combine fixed inventories without duplicates.

intersection()

Returns values common to both.

```
prod = frozenset({"web01", "db01"})
critical = frozenset({"db01", "db02"})
common = prod.intersection(critical)
print(common)
```

Real-time use

Find common servers, permissions or security findings.

difference()

Returns values in the first frozenset but not the second.

```
old = frozenset({"web01", "web02", "db01"})
new = frozenset({"web01", "web02"})
removed = old.difference(new)
print(removed)
```

Real-time use

Compare fixed snapshots and identify differences.

symmetric_difference()

Returns values in either set, but not both.

```
a = frozenset({"AWS", "Azure", "Docker"})
b = frozenset({"Azure", "Kubernetes", "Docker"})
different = a.symmetric_difference(b)
print(different)
```

Real-time use

Find non-common capabilities, technologies or configuration values.

issubset()

Checks whether all values of one frozenset are inside another.

```
required = frozenset({"HTTPS", "SSH"})
allowed = frozenset({"HTTP", "HTTPS", "SSH"})
print(required.issubset(allowed))
```

Real-time use

Validate whether required permissions/features are allowed.

issuperset()

Checks whether one frozenset contains all values of another.

```
allowed = frozenset({"HTTP", "HTTPS", "SSH"})
required = frozenset({"HTTPS", "SSH"})
print(allowed.issuperset(required))
```

Real-time use

Validate a fixed access-policy set.

isdisjoint()

Checks whether two frozensets have no common values.

```
prod = frozenset({"web01", "db01"})
test = frozenset({"web10", "db10"})
print(prod.isdisjoint(test))
```

Real-time use

Check whether two fixed resource groups overlap.

copy()

Returns another frozenset with the same values.

```
approved = frozenset({"read", "write"})
backup = approved.copy()
print(backup)
```

Real-time use

Keep a separate reference/copy of an immutable permission set.

__contains__ / in

Membership check using the in operator.

```
allowed_ports = frozenset({80, 443})
port = 443
print(port in allowed_ports) # True
```

Real-time use

Useful for fixed allow-lists such as ports, users or permissions.

5. Frozenset Operators

```
a = frozenset({"web01", "web02"})
b = frozenset({"web02", "db01"})

print(a | b) # union
print(a & b) # intersection
print(a - b) # difference
print(a ^ b) # symmetric difference
```

Remember

| UNION & INTERSECTION - DIFFERENCE ^ SYMMETRIC DIFFERENCE

6. Frozenset as a Dictionary Key ■

A normal set cannot be a dictionary key because it is mutable. A frozenset is immutable and hashable, so it can be used as a key.

```
permissions = frozenset({"read", "write"})
role_map = {
    permissions: "Developer"
}

print(role_map[permissions]) # Developer
```

Real-time security example

A fixed combination of permissions can be represented by a frozenset and safely used as a dictionary key for role/configuration mapping.

7. Set vs Frozenset

Feature	set	frozenset
Unique values	■	■
Mutable	■ Yes	■ No
add/update	■	■
remove/pop/clear	■	■
Set operations	■	■
Hashable	■	■
Dictionary key	■	■
Use case	Data changes	Fixed/immutable set

8. Real-Time DevOps / Security Examples

Fixed Allowed Ports

```
allowed_ports = frozenset({22, 80, 443})
incoming_port = 443

if incoming_port in allowed_ports:
    print("Port allowed")
else:
    print("Port blocked")
```

Immutable IAM Permissions

```
admin_permissions = frozenset({
    "read", "write", "delete", "manage_users"
})

required = frozenset({"read", "write"})

print(required.issubset(admin_permissions)) # True
```

Compare Two Fixed Environments

```
prod = frozenset({"HTTPS", "WAF", "MFA"})
dr = frozenset({"HTTPS", "WAF"})

missing_in_dr = prod.difference(dr)
print(missing_in_dr) # {'MFA'}
```

Remove Duplicate Security Findings

```
findings = ["CVE-101", "CVE-102", "CVE-101", "CVE-103"]
unique_findings = frozenset(findings)
print(unique_findings)
```

9. Interview Questions ■

- *What is frozenset in Python?*
- *Difference between set and frozenset?*
- *Why is frozenset immutable?*
- *Can we use add() on a frozenset? — No.*
- *Can a frozenset contain duplicate values? — No.*
- *Can a frozenset be used as a dictionary key? — Yes.*
- *Why can a normal set not be a dictionary key? — It is mutable and unhashable.*
- *Which set operations work with frozenset? — union, intersection, difference, symmetric_difference and comparison operations.*
- *How do you create an empty frozenset? — frozenset().*
- *When would you use frozenset in real-time systems? — For fixed, unique collections that should not change.*

10. Practice Labs ■

- *Create a frozenset of approved server names and test membership.*

- Create two frozensets of ports and find common ports using `intersection()`.
- Compare production and DR frozensets and find missing security controls.
- Create a frozenset of IAM permissions and check a required permission set with `issubset()`.
- Use a frozenset as a dictionary key.
- Convert a list containing duplicate IP addresses into a frozenset.
- Try `add()`, `remove()` and `pop()` on a frozenset and observe the errors.
- Compare set, tuple and frozenset for mutable vs immutable behavior.

11. Quick Reference

Method / operation	Purpose
<code>union()</code>	Combine unique values
<code>intersection()</code>	Common values
<code>difference()</code>	Only values in first
<code>symmetric_difference()</code>	Values in either, not both
<code>issubset()</code>	Check subset
<code>issuperset()</code>	Check superset
<code>isdisjoint()</code>	Check no overlap
<code>copy()</code>	Return a frozenset copy
<code>in</code>	Membership check
<code> </code> , <code>&</code> , <code>-</code> , <code>^</code>	Set operators

■ Final Rule

`frozenset` = unique + unordered + immutable + hashable.

It supports set comparison/combination operations, but not methods that modify the collection.